

IE3090
Network Programming
3rd Year, Semester I

Prompt Log — IE3090 RemoteOps

Submitted to

Sri Lanka Institute of Information Technology

In partial fulfillment of the requirements for the

Bachelor of Science Special Honors Degree in Information Technology

5th October 2026

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Registration Number : IT24101899

Name : Kavirathna N R

INTRODUCTION

I used AI tools to assist with Part 1 of the Remote Ops assignment, as permitted under Section 3 of the brief (CLEAR Level 3: AI Collaboration). The tools I used were Claude, DeepSeek, and ChatGPT. I critically evaluated, tested, and modified all AI output before including it in my submission. Every AI-generated piece of code was read, understood, and where necessary rewritten by me. This log records the substantive interactions across multiple sessions during the assignment period.

ENTRY 1 — Understanding the socket API

Tool: Claude

Stage: Early design

Prompt:

"Explain how socket(), bind(), listen(), accept() work together in a TCP server, with a simple C example."

Output received:

A step-by-step explanation of the TCP connection lifecycle with a single-client echo server example.

How I used it:

I used this to understand the standard Berkeley sockets workflow.I then extended the single-client example into a thread-per-connection model (using pthread_create and pthread_detach) to satisfy the concurrency requirement in §2.2.1 of the brief.

Modifications:

- Added thread-per-connection instead of the single-client model suggested

- Personalised all constants (AGENT_PORT 9410, SID "9981", TOKEN "OPS-1899", LOG_FILE, STORE_DIR)

- Added SO_REUSEADDR handling to avoid bind() failures on restart

ENTRY 2 — Writing boilerplate TCP server code

Tool: Claude, ChatGPT

Stage: Initial implementation of agent_899.c

Prompt:

"Give me a C skeleton for a TCP server that handles multiple clients with one thread per connection, using pthreads."

Output received:

A basic skeleton with socket/bind/listen/accept loop and a client_thread function.

How I used it:

I used this as the starting point for agent_899.c. I then added:

Modifications:

- Per-session state in a session_t struct (fd, peer address,

  authed flag, rxbuf, monitoring state)

- Personalised constants at the top of the file

- Authentication check before any command except AUTH

- The full command dispatch (handle_line)

- SIGPIPE ignored with MSG_NOSIGNAL on all send() calls

- Timestamped logging with logmsg()

The final agent_899.c is significantly larger than the AI's skeleton and was written by me on top of it.

ENTRY 3 — Debugging the Makefile "missing separator" error

Tool: DeepSeek

Stage: Build setup

Prompt:

"make says 'missing separator. Stop.' on my Makefile. Why?"

Output received:

Explanation that Makefile recipe lines must begin with a real TAB character, not spaces. Suggested using cat -A to check.

How I used it:

I ran cat -A Makefile_899 and confirmed the recipe lines started with spaces instead of ^I (TAB). I re-typed those lines in nano using real tabs. The build then worked.

Modifications:

- Verified with cat -A as suggested

- Corrected the file using nano (not the AI's method)

ENTRY 4 — Debugging the #include <stdarg.h> missing error

Tool: Claude

Stage: Compiling agent_899.c

Prompt:

"gcc says 'va_list undeclared'. What do I need to include?"

Output received:

Explanation that va_list, va_start, va_end, and vfprintf require #include <stdarg.h>, and that I was using them in logmsg() and reply() for variadic arguments.

How I used it:

I added #include <stdarg.h> to the top of agent_899.c. Recompiled successfully.

Modifications:

- Directly applied the fix

- This was my first experience with variadic functions

ENTRY 5 — Reading /proc files for SYSINFO

Tool: Claude

Stage: Implementing do_sysinfo()

Prompt:

"How do I read CPU load, memory usage, and uptime from /proc in C?"

Output received:

Suggested fopen/fscanf approach for /proc/loadavg (1-minute load), /proc/meminfo (MemTotal - MemAvailable), and /proc/uptime.

How I used it:

I implemented get_sysinfo() using these three files.

Modifications:

- Wrote my own parser for /proc/meminfo that scans for specific keys (MemTotal,MemAvailable)

- Added fallback to 0 if the file cannot be opened

- Converted kB to MB by dividing by 1024

ENTRY 6 — The stty -echo mistake that broke my terminal

Tool: Claude

Stage: During screenshotting

Prompt:

"How can I stop the terminal from echoing while I take a

screenshot?"

Output received:

Suggested `stty -echo` (which disables character echo).

What went wrong:

This broke my interactive terminal — I could type but could not see any characters. Commands got merged with previous text and I could not navigate.

How I recovered:

I searched for a fix, ran `reset`, and the terminal returned to normal. Lesson learned: never disable echo for interactive sessions.

Modifications:

- Rejected this approach

- Instead, I slowed the UDP interval (see Entry 7) to make

  screenshots readable

ENTRY 7 — UDP flooding during screenshots

Tool: Claude

Stage: Taking screenshots of the UDP monitoring stream

Problem:

The Agent's monitor_thread was sending UDP datagrams every 2 seconds, which flooded the Controller terminal and made it impossible to type MONITOR STOP cleanly for a screenshot.

AI suggestion:

Wait for 2–3 datagrams, then type the stop command quickly. Or temporarily change MON_INTERVAL_SEC in agent_899.c to a larger value.

What I did:

I changed #define MON_INTERVAL_SEC from 2 to 5 in agent_899.c, rebuilt, and was then able to take clean screenshots showing MONITOR START, several datagrams, and MONITOR STOP.

Modifications:

- Changed the interval from 2 to 5 seconds

- Noted this change in §8.2 of the Implementation Report

- This was my decision, not the AI's default

ENTRY 8 — Debugging SSH disconnect during session

Tool: Claude

Stage: Recovering work after a dropped connection

Problem:

During one of the sessions, my SSH connection to the CentOS server was reset, killing the running Agent process.

AI suggestion:

Reconnect with ssh, verify files were safe on disk, restart the Agent, and in future use tmux to keep sessions alive across disconnects.

How I used it:

- Reconnected successfully
- Confirmed all files (agent_899.c, controller_899.c, Makefile_899,
  log file, storage folder) survived the disconnect
- Restarted the Agent
- Used tmux going forward to keep sessions persistent

Modifications:

- Adopted tmux for long-running sessions

ENTRY 9 — Understanding what the assignment was asking for

Tool: Claude, DeepSeek

Stage: Reading the brief

Prompt:

"Explain §2.4 personalisation. What does 'last four digits reversed' mean for registration IT24101899?"

Output received:

Walkthrough of each personalised value:

- Port = 7000 + first four digits (2410) = 9410

- Source files = agent_899.c, controller_899.c, Makefile_899

- SID = last four digits (1899) reversed = 9981

- Token = "OPS-" + last four digits = OPS-1899

- Log file = remoteops_IT24101899.log

- Storage path = ./agentfiles/IT24101899/

How I used it:

Verified each value manually. Hard-coded them as named constants at the top of agent_899.c and controller_899.c so they are defined in one place.

Modifications:

- Double-checked every value against my registration number

- Ensured consistency across both source files, the log, and the report

ENTRY 10 — Aligning function names between code and report

Tool: Claude

Stage: Finalising the Implementation Report

Prompt:

"My report describes functions like logmsg(), reply(), do_sysinfo(), recv_bytes(). My code uses different names. How do I make them consistent?"

Output received:

Suggested renaming my functions to match the report's wording (logmsg instead of log_event, reply instead of send_ok/send_err,recv_bytes instead of read_exact, etc.).

How I used it:

Renamed all functions in agent_899.c and controller_899.c to match the report. This ensures the Viva examiner sees the same names in both places.

Modifications:

- Manual rename throughout both source files

- Verified with grep -nE '^(static|int|void)'

ENTRY 11 — Writing the Implementation Report structure

Tool: Claude

Stage: Report writing

Prompt:

"What sections does the IE3090 RemoteOps Implementation Report need?"

Output received:

A section outline matching §2.7 of the brief.

How I used it:

Used the outline as a skeleton, then wrote my own prose for every section based on my actual code, tests, and screenshots. No AI-generated prose was submitted verbatim.

Modifications:

- Wrote all prose myself
- Added actual screenshots from my own runs
- Added a testing summary reflecting tests I actually performed

SUMMARY

Tools used:

- Claude (primary)

- DeepSeek (secondary, for Makefile and compilation issues)

- ChatGPT (occasionally, for alternative explanations)

Stages where AI was used:

- Understanding socket programming and the TCP lifecycle

- Writing the initial boilerplate for the Agent and Controller

- Debugging compilation errors (missing includes, Makefile tabs)

- Understanding /proc file parsing for SYSINFO

- Debugging the terminal state after stty -echo

- Understanding the assignment brief and personalisation rules

- Aligning code and report function names

Where AI was helpful:

- Explaining socket API concepts clearly

- Debugging specific error messages quickly

- Providing a starting skeleton that I could extend

- Suggesting tmux for persistent sessions

- Helping understand the personalisation formulas

Where AI was wrong or misleading:

- Suggested `stty -echo` which broke my terminal (I had to run `reset` to recover)

- The initial agent_899.c skeleton was missing #include <stdarg.h> for the variadic logging function

- Suggested a 2-second UDP monitoring interval, which caused flooding during screenshots — I changed it to 5 seconds

What I changed, added, or rejected:

- All personalised values derived from my own registration number
- Changed the monitor interval from 2 s to 5 s
- Renamed functions to match the report
- Rejected stty -echo as a screenshot technique
- Rewrote the concurrency explanation based on my own understanding of why threads were chosen over fork/select
- Wrote all report prose and captions myself

Overall reflection on AI usage:

AI was useful as a learning aid and for debugging specific errors,but the design decisions (thread-per-connection model, session_t structure, personalised constants, whitelist handling) were mine. Every command was tested manually against a real Agent and Controller on the CentOS server. The Viva will confirm my understanding of the code.

END OF PROMPT LOG